



JAVA

TP Lido Serena

Contexte

Système de gestion de caisse d'un restaurant

Le Lido Serena, restaurant de plage italien vous demande de créer le **système de caisse** de son restaurant

Le restaurant a chargé une autre équipe de créer une application de prise de commande

Actuellement quand une commande est prise on l'enregistre dans un fichier json nommé *numéroTable.json* (**1.json**, **2.json** ...)

Contexte

Le système de caisse aura plusieurs objectifs :

- **Afficher les détails** des produits pour informer les clients
- Faire **payer** les clients qui ont fini de manger
- Proposer une caisse de secours en cas de panne de l'application de prise de commande
- **Archiver** les paiements pour en faire des stats plus tard

Contexte

- Le Lido Serena à **3 types de produits**
 - *Les plats*
 - *Les boissons*
 - *Les desserts*
- Les 3 ont des **choses en communs**
 - **Nom**
 - **Prix**
 - **Description**
 - On peut vouloir connaître le détail de chaque produit (méthode **afficher_détail**)
- Mais chacun à des **spécificités**
 - Un plat à un **type** (*pizza, pasta, secondi*)
 - Une boisson à un **volume** (*12cl, 33cl, 50cl, 1L ...*)
 - Les desserts affichent leurs **calories**
- Toutes les spécificités doivent être **afficher dans le détail** du produit

Contexte

Votre *tech lead* vous demande d'utiliser la **POO** dans ce projet pour avoir un code maintenable et réutilisable

Il détecte au moins 3 objets nécessaires :

- ***Produit***
- ***Payement***
- ***Commande***

Mais la diversité des produits lui fait dire qu'il faudra probablement créer d'autres objets avec les concepts d'**héritage et de polymorphisme**.

Contexte

Il vous conseil également de prévoir des **constructeur** à partir de ***JSONObject*** pour générer facilement des instance de produit à partir des fichier *JSON*

Ainsi que des **méthodes** permettant de générer un ***JSONObject*** depuis une instance de commande pour le stocker dans le **fichier d'archive**

Réalisation

Au démarrage de l'application le caissier aura 4 possibilités

1. Afficher les détails d'un produit
2. Réaliser le paiement d'une table
3. Utiliser la caisse de secours
4. Quitter le programme

Réalisation

1. Afficher les détails d'un produit

- **Afficher** la liste de tous les produits avec un numéro (1 - Margherita)
- **Demander** au caissier sur quel produit il souhaite des informations
 - On lui demande de saisir **le numéro** pour éviter les erreurs
- On affiche les informations du produits proprement à travers la méthode ***afficher_details*** du produit

Réalisation

2. Réaliser le paiement d'une table

- **Demander** au caissier quelle table souhaite payer
- Trouver le fichier json de la commande de cette table grâce à son numéro
- **Récupérer** les produits commandés
- Récupérer le prix de chacun de ces produits dans le fichier ***products.json***
- **Calculer l'addition**
- **Stocker la commande** payée dans le fichier ***archive.json***



On veut dans l'archive toutes les informations des produits

Réalisation

3. Utiliser la caisse de secours

- **Demander** au caissier les informations de la tables (*numéro, nb de pers*)
- Demander au caissier ce que la table a commandé en lui **affichant les produits** (il doit **choisir un numéro** parmi une liste)
- **Recommencer** jusqu'à ce que le caissier arrête la saisie des produits
- Calculer l'addition
- Stocker la commande payée dans le fichier **archive.json**



On veut dans l'archive **toutes les informations des produits**

Règles

- Cours et internet autorisé
- IA autorisé si vous comprenez ce que vous faites (**sauf pendant l'oral**)
- Autocomplétion désactivé pour l'oral
- Bonus et Malus possibles
- Note de 0 à 20
- Déposez votre projet sous au format compressé nommé **nom_prenom** [ICI](#)

Conseil

- **Prenez votre temps**
- **Comprenez** ce que vous faites
- Ne faites pas un gros copier coller chatGPT
- Comprenez **vos erreur**
- Commencez par créer les classes **dont vous avez besoins** (produits)